

Stage 2: JAX for Robotics

JAX used for parallel policy training (thousands of robots)

- traces Python into computation graph and compiles it, comes with certain constraints

no branching on traced values

JAX arrays are immutable

no loops dependent on traced runtime values,

use JAX control flow constructs

quaternion conventions: MuJoCo uses wxyz

ROS uses xyzw

Part 1: `pop/jaxlab/ex1-pure-functions.py`

- use `jnp.where(x > 0, x, -x)` instead of `if (x > 0)`

No python control flow

on traced values

- use `values.at[index].set(value)` instead of
`values[index] = value`

No mutating arrays, this returns
a new array instead

- use python loop with constant bound or
`jax.lax.fori-loop` instead of `for i in range(n)`

No python loops with traced bounds

`return jax.lax.fori-loop(0, count, lambda i, total: total + i, 0)`

start total at 0, for every i from 0 up to $\text{count} - 1$,
replace total with $\text{total} + i$, and return total when done

Part 2: `pyp/jaxlab/ex2_vmap_pd.py`

no changing shapes or dtypes

`jnp.clip` → forcing torque to be in $\pm 20 \text{ Nm}$ range

`vmap` → creating vectorized function

`Os` → take input one row at a time along axis 0

`Nave` → give every robot the same k_p and k_d

Part 3: `pup/jaxlab/ex3_quaternions.py`

MuJoCo convention is $wxyz$

ROS convention is $xyzw$

`quat_rotate(q, v) → rotate (3,) by unit  $wxyz$  (4,)`

rotate 3D vector by rotation present in quaternion
through sequential cross products

`quat_inv(q) → inverse of a unit quaternion`

reversing direction of all fields of quaternion

`gravity_in_body_frame(q_wb) → the world`

down-direction $[0, 0, -1]$ expressed in the body frame

What direction is gravity relative to the robot's body